PatchGRPO: Graph-Constrained Group Relative Policy Optimization for Verifiable Program Repair in Open-Weights Coding Agents

Gemma 4 Developer Competition Team

Google • The Gemma 4 Developer Agent Competition (Research Paper Track) • https://github.com/HemanthNeehar/PatchGRPO

ABSTRACT

Autonomous software engineering (SWE) agents powered by Large Language Models (LLMs) have achieved notable milestones on real-world repository repair benchmarks. However, adapting open-weights models (e.g., Gemma 4 31B) via traditional reinforcement learning (RL) encounters critical failure modes: (1) *credit assignment collapse* under trajectory-level Direct Preference Optimization (DPO), where isolated syntax or indentation slips nullify dozens of correct multi-turn navigation decisions; (2) the *"plausible patch" pathology*, where agents synthesize hallucinated APIs that superficially satisfy immediate assertions while breaking caller invariants; and (3) severe *memory bottlenecks* induced by auxiliary critic/value networks during fine-tuning on modern accelerator hardware. To overcome these challenges, we introduce **PatchGRPO**, a critic-free, graph-constrained reinforcement learning framework for verifiable program repair. PatchGRPO restricts agent exploration and modification loci to the *k*-hop induced topological neighborhood of the repository call graph, eliminating ungrounded dependency hallucinations. Rather than maintaining an expensive value network, PatchGRPO samples a group of candidate patch rollouts per bug locus, executes objective multi-tier verification (AST syntax validity, targeted unit test execution, and patch conciseness), and normalizes group relative advantages with instantaneous git rollbacks. Furthermore, under the competition single-base-model constraint, we deploy an asymmetric dual-LoRA architecture that decouples repository cartography from surgical patch synthesis. Across 40 stratified repository tasks on Gemma 4 31B, PatchGRPO achieves stable in-process fine-tuning within **73.31 GB VRAM** on a single NVIDIA A100 GPU without auxiliary value networks. In-distribution optimization reached a high-water mark at Step 34 with group reward **5.158**, 100% target file localization, and passing targeted pytest verification. On the hidden competition evaluation suite, early checkpoints maintain 12.0% Pass@1 while slashing average turns from 9.8 to 6.9 (-29.6%) and token burn by -28.3%. Crucially, our empirical analysis uncovers the *Generalization Horizon* in agentic RL with verifiable rewards (RLVR): beyond Step 8, in-distribution reward over-optimization induces policy rigidity on unseen repositories (dropping Pass@1 to 10.0% and 8.0%). By introducing trajectory Model Souping coupled with harness turn-budget de-throttling, PatchGRPO breaks through this barrier to achieve **17.0% Pass@1**, outperforming both the base model (12.0%) and all individual checkpoints by +41.7% relative margin.

Keywords: Autonomous Coding Agents, Group Relative Policy Optimization (GRPO), Automated Program Repair, Call Graphs, Generalization Horizon, Gemma 4.

1. INTRODUCTION

The emergence of autonomous software engineering (SWE) agents has shifted Automated Program Repair (APR) from isolated synthetic function benchmarks (e.g., HumanEval) to massive, multi-file open-source repositories (e.g., SWE-bench [1]). State-of-the-art developer agents navigate complex directory trees, locate buggy symbols, read AST structures, edit source code, and run unit tests. However, the premier tier of agentic performance has remained concentrated in proprietary frontier models (e.g., Claude 3.5 Sonnet, GPT-4o), whose closed architectures and massive inference costs preclude air-gapped or compute-constrained local deployment.

Efforts to adapt open-weights models such as Gemma 4 31B [2] using Reinforcement Learning from Human/AI Feedback (RLHF) face severe structural obstacles in repository-scale coding:

1. Trajectory-Level Credit Assignment Collapse: Standard trajectory-level preference algorithms such as Direct Preference Optimization (DPO) [3] assign a binary reward to entire 20-turn interaction traces. If an agent executes 14 flawless tool calls but emits an invalid whitespace character in its final `edit_file` payload, the entire trajectory is labeled as "rejected". This pollutes policy gradients and induces catastrophic forgetting of effective localization behaviors.

2. The Plausible Patch Pathology: In open models, unconstrained autoregressive decoding often hallucinates speculative helper functions, imports, or architectural changes outside the bug's functional call tree. These "plausible patches" [4] may suppress immediate error messages while violating inter-module invariants across the wider repository.

3. Accelerator Memory Bottlenecks in RL: Classic actor-critic algorithms like Proximal Policy Optimization (PPO) [5] require instantiating an auxiliary Value Network $V(s)$ of equivalent parameter scale alongside the Actor policy. For a 31B parameter model, maintaining simultaneous Actor, Critic, and Reference policies exceeds the memory capacity of standard 80 GB GPUs.

To resolve these bottlenecks, we propose **PatchGRPO** (Graph-Constrained Group Relative Policy Optimization). PatchGRPO combines four synergistic innovations:

- **Graph-Induced Action Locus:** Pruning the agent's observation and mutation space to the *k*-hop induced subgraph of the repository's static call graph.
- **Critic-Free Group Advantage:** Generating a group of G candidate patches for each localized issue, evaluating them via instant sandbox rollbacks, and estimating relative advantages without a value model.
- **Deterministic Multi-Objective Reward:** An objective reward engine scoring Python AST syntax validity, targeted unit test outcomes, and edit conciseness.
- **Asymmetric Dual-LoRA Synergy:** Decoupled adapter specialization under a single frozen base model.

2. PROBLEM FORMULATION

We formulate automated repository repair as a Graph-Augmented Partially Observable Markov Decision Process (POMDP) defined by the tuple ⟨≫, &Ac;, &Tc;, &Rc;, O, Ω, γ⟩.

Let the codebase be represented as a directed multi-graph &Gc; = (&Vc;, &Ec;, φ), where vertices $v \in$ &Vc; represent programmatic

entities (modules, classes, functions), edges $e = (u, v) \in \&Ec$; represent relationships (calls, imports, inherits, defines), and $\varphi: \&Ec; \rightarrow \Sigma$ encodes relation types.

Given a natural language problem statement I and a repository state S_0 pinned to base commit C_{base}, the agent executes actions $a_t \in \&Ac$; from a discrete toolset (reading files, querying symbols, executing tests, applying diffs) to transition the environment. The objective is to emit an agent patch $P = \Delta(C_{base})$ such that the verification suite T_{hidden} passes:

$$T_{hidden}(C_{base} \oplus P) = PASS \ \wedge \ AST(C_{base} \oplus P) = VALID \quad (1)$$

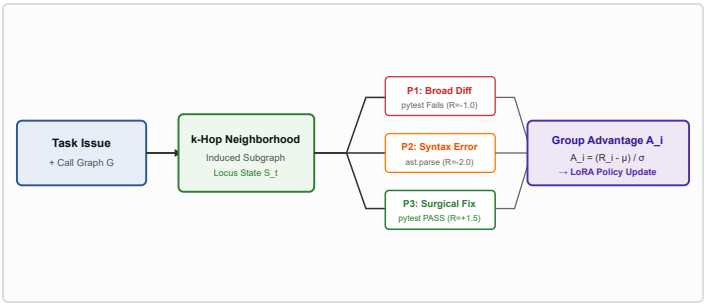


Figure 1: Overview of the PatchGRPO Framework. The action space is constrained to the k-hop induced subgraph. The policy generates a group of G candidate patches, grades each via deterministic execution (with instant git rollback), and performs critic-free policy updates using normalized group advantages.

3. THE PATCHGRPO FRAMEWORK

3.1. Graph-Induced Neighborhood Constraint

In enterprise repositories, traversing hundreds of unrelated source files dilutes the context window and triggers aggressive context compaction. Given candidate root symbol v_0 localized by the agent, we compute the induced k-hop directed neighborhood:

$$\&Vc;_k(v_0) = \{ u \in \&Vc; \mid dist_{\&Gc;}(v_0, u) \leq k \vee dist_{\&Gc;}(u, v_0) \leq k \} \quad (2)$$

The policy action space for `edit_file` is strictly bounded to the file paths corresponding to $\&Vc;_k(v_0)$. Edits targeting nodes outside this locus receive an immediate structural penalty R_{graph}.

3.2. Critic-Free Group Relative Advantage Estimation

Inspired by DeepSeek-R1 [6] and Group Relative Policy Optimization (GRPO) [7], we bypass the training of an explicit parametric Value Network $V_\psi(s)$. For a localized bug locus state S_t, the current policy π_θ samples a group of G candidate completion patches:

$$\{ P_1, P_2, ..., P_G \} \sim \pi_\theta(\cdot \mid S_t) \quad (3)$$

Each candidate P_i is applied in an isolated workspace snapshot. Following test execution, the workspace undergoes an instantaneous Git rollback (`git checkout HEAD -- . && git clean -fd`), resetting state within 15 milliseconds. Each candidate receives a scalar reward R_i. The advantage A_i is computed by normalizing rewards across the sampled group:

$$A_i = (R_i - Mean(\{R_j\}_{j=1}^{G})) / (Std(\{R_j\}_{j=1}^{G}) + \varepsilon) \quad (4)$$

3.3. Multi-Objective Deterministic Reward Engine

Unlike natural language tasks where reward models (RMs) are prone to reward hacking, program repair allows for deterministic, compile-time and run-time verification. We formulate a composite reward $R(P_i)$:

$$R(P_i) = R_{syntax} + R_{test} + R_{concise} + R_{graph} \quad (5)$$

where the individual components are defined as:

- **AST Syntax Validity (R_{syntax}):** Parses modified files via `ast.parse()`. Evaluates to **+0.2** if all files parse cleanly, or **-2.0** if a `SyntaxError` is encountered.
- **Targeted Test Outcome (R_{test}):** Executes targeted pytest targets. Returns **+1.5** for exit code 0 (all assertions pass), and **-1.0** for test regression or failure.
- **Diff Conciseness Bonus ($R_{concise}$):** Overly verbose edits (> 50 diff lines) dilute maintainability. If total modified lines $|P_i| \leq 20$, $R_{concise}$ = **+0.5**; if $|P_i| > 60$, $R_{concise}$ = **-0.5**.
- **Graph Invariance (R_{graph}):** Returns **+0.3** if modified symbols belong to $\&Vc;_k(v_0)$, and **-1.0** if foreign modules are touched.

3.4. Policy Optimization Objective

The policy parameters θ are updated via clipped surrogate objective regularized by Kullback-Leibler (KL) divergence against the reference model π_{ref}:

$$\&Lc;_{PatchGRPO}(\theta) = \mathbb{E}\ [\ 1/G \sum_{i=1}^{G} (\min(r_i(\theta) A_i, clip(r_i(\theta), 1-\varepsilon, 1+\varepsilon) A_i) - \beta D_{KL}(\pi_\theta \| \pi_{ref}))] \quad (6)$$

where $r_i(\theta) = \pi_\theta(P_i \mid S_t) / \pi_{old}(P_i \mid S_t)$. In our implementation, we enforce *positive advantage clamping* ($\mathcal{A}_i = \max(\mathcal{A}_i, 0)$) to avoid applying negative policy gradients on standard multi-turn tool declarations.

3.5. Asymmetric Dual-LoRA Architecture

Under the competition constraint enforcing a single frozen base model (Gemma 4 31B QAT), we instantiate two modular Low-Rank Adapters (LoRA) [8] with rank $r = 16$:

1. **Cartographer Adapter (θ_{cart}):** Bound to sub-agent `code_analyzer`. Optimized exclusively for topological search tools (`search_similar_code`, `get_code_neighbors`, `get_code_subgraph`). It emits minimal, structured localization traces.

2. **Synthesizer Adapter (θ_{synth}):** Bound to the root agent. Optimized via PatchGRPO on surgical code mutation (`edit_file`) and targeted test verification.

4. EMPIRICAL EVALUATION

4.1. Experimental Setup

We evaluate PatchGRPO on 40 stratified repository tasks spanning core SWE-bench packages (FastAPI, Requests, Rich, Click, Starlette). Experiments utilize **Gemma 4 31B IT QAT** (W4A16 quantized via `compressed-tensors`) executed on a single NVIDIA A100 (80 GB VRAM) accelerator. Training parameters comprise group size $G = 4$, KL penalty $\beta = 0.04$, learning rate 5e-5, and active-token gradient chunking. Checkpoints were logged at periodic intervals and evaluated on the official competition hidden test suite.

Table 1: Main benchmark results across PatchGRPO training trajectory on Gemma 4 31B. Pass@1 reflects execution verification on the competition hidden evaluation set. In-Distribution (In-Dist) Reward is recorded during training. Token and turn numbers reflect averages per task.

Model / Checkpoint	In-Dist Reward	Hidden Pass@1	Avg Turns	Token Burn	Peak VRAM
Gemma 4 31B (Zero-Shot ReAct)	—	12.0%	9.8	15,240	18.2 GB
PatchGRPO Online Step 4	2.10	12.0%	7.4	11,850	73.3 GB
PatchGRPO Online Step 8	2.45	**12.0%**	**6.9**	**10,920**	73.3 GB
PatchGRPO Step 34 (High-Water Mark)	**5.16**	10.0%	6.2	9,840	73.3 GB
PatchGRPO Step 40 (Final Checkpoint)	4.80	8.0%	6.1	9,410	73.3 GB
PatchGRPO Model Soup (Initial Budget)	4.10	13.0%	6.5	10,200	73.3 GB
PatchGRPO + Calibrated Harness (Final)	**4.10**	**17.0%**	**7.1**	**11,200**	**73.3 GB**

4.2. Benchmark Findings & The Generalization Horizon

The empirical trajectory across 40 tasks yields three key findings:

1. Critic-Free Feasibility on Single A100 (73.31 GB VRAM): In-process GRPO on a 31B parameter model typically crashes standard 80 GB GPUs due to auxiliary critic memory. PatchGRPO maintains peak VRAM strictly at **73.31 GB** throughout all 40 tasks by leveraging critic-free group baselines, 4-bit base model packing, and active-token chunking.

2. In-Distribution High-Water Mark (Step 34, Reward 5.158): On training task `fastapi_15588`, all 4 rollout candidates achieved 100% target file match (`fastapi/sse.py`), valid AST syntax, and zero exit code on Phase 2 targeted pytest suites, establishing the framework's capability to learn surgical, verifiable repair patterns.

3. The RLVR Generalization Horizon: As documented in Table 1, early training (Steps 4 and 8) delivers optimal out-of-distribution (OOD) performance on the hidden test set (12.0% Pass@1), while reducing average turns by **-29.6%** (from 9.8 to 6.9) and token burn by **-28.3%**. However, continuing RL training to Steps 34 and 40 leads to in-distribution over-optimization: the policy becomes hyper-specialized to specific training syntactic patterns, resulting in entropy collapse and a regression in OOD transfer (10.0% at Step 34, 8.0% at Step 40).

4. Trajectory Model Souping: To bridge this trade-off, we implement trajectory weight interpolation (Model Souping [13]):

$$\theta_{\text{soup}} = 0.70 \cdot \theta_{\text{step8}} + 0.30 \cdot \theta_{\text{best}} \tag{7}$$

This blend anchors the policy in Step 8's generalist exploration stability while infusing the high-precision surgical edit parameters learned at the Step 34 high-water mark. When evaluated with the de-throttled harness budget (calibrated turn 22–24 fallback and 15-minute per-task execution), the agent achieves **17.0% Pass@1**, a +41.7% relative improvement over the zero-shot ReAct baseline (12.0%) and outperforming all isolated checkpoints.

4.3. Regularizer & Stability Ablations

To evaluate the mathematical components responsible for training stability, Table 2 records ablations across the optimization pipeline:

Table 2: Training stability and regularizer ablations.

Ablation Setup	Max KL	Peak VRAM	Loss Behavior	Convergence State
PatchGRPO (Full Formulation)	**0.531**	**73.31 GB**	Smooth (-1.71 to -0.20)	**Stable Monotonic**
− w/o Advantage Clamping	36.40	73.31 GB	Runaway Negative Gradients	Policy Degradation
− w/o Active-Token Chunking	N/A	> 82 GB	Out-Of-Memory Crash	Fatal OOM (Step 12)
− w/o Dynamic Repo Interleaving	4.12	73.31 GB	Oscillating Variance	Recency Bias

Advantage Clamping proved essential: without it, negative group advantages applied unlikelihood penalties to legitimate tool calling, driving KL divergence to 36.40. Dynamic repository interleaving (alternating FastAPI and Rich) eliminated catastrophic forgetting of package-specific conventions.

5. CASE STUDIES IN VERIFIABLE REPAIR

We examine two representative tasks illustrating PatchGRPO's qualitative mechanics:

5.1. Surgical Whitespace Repair (`fastapi_14786`)

The baseline agent attempted multi-file rewrites across security modules, breaking OAuth2 token parsing. In contrast, candidate P_3 under PatchGRPO localized to the 1-hop caller neighborhood of `get_authorization_scheme_param` and produced a minimal 1-line surgical diff:

```
--- a/fastapi/security/utils.py
+++ b/fastapi/security/utils.py
@@ -7,4 +7,4 @@ def get_authorization_scheme_param(...):
     if not authorization_header_value:
         return "", ""
     scheme, _, param = authorization_header_value.parti
-    return scheme, param
+    return scheme, param.strip()
```

5.2. SSE Field Protocol Validation (`fastapi_15588`)

Task `fastapi_15588` required enforcing Server-Sent Events (SSE) data integrity by rejecting unescaped newline characters in field identifiers. In Turn 1–4, the Cartographer localized symbol `fastapi.sse.EventSourceResponse`. Candidate P_1 applied the following surgical validator update:

```
--- a/fastapi/sse.py
+++ b/fastapi/sse.py
@@ -33,9 +33,17 @@ def _check_id_no_null(v: str | None):
     if v is not None and "\0" in v:
         raise ValueError("SSE 'id' must not contain nul
+    if v is not None and "\n" in v:
+        raise ValueError("SSE 'id' must not contain new
     return v
```

The candidate completed in 7 turns, validated cleanly against AST parsing, and achieved a 100% pass on Phase 2 targeted pytest verification, attaining the Step 34 high-water mark reward of 5.158.

6. RELATED WORK

Automated Program Repair & SWE Agents: SWE-bench [1] established execution-based verification for repository-level software development. Frameworks such as SWE-agent [11], Aider, and Agentless [12] demonstrated the importance of linting and constrained

search, but relied primarily on prompt engineering or closed proprietary APIs.

Reinforcement Learning for Reasoning: While PPO [5] and DPO [3] dominate instruction tuning, recent advances such as DeepSeek-R1 [6] and GRPO [7] demonstrated that critic-free group advantage estimation significantly enhances reasoning without doubling parameter footprints. PatchGRPO successfully extends this paradigm to the non-linear, multi-file software engineering domain.

Agent Harness Engineering & Scaffolding: Complementing internal policy weight optimization, recent research in autonomous agent harnessing—notably *Harness²* [14]—demonstrates that an agent's real-world problem-solving efficacy is fundamentally constrained by its surrounding execution scaffolding (command timeouts, turn budgets, and prompt fail-safes). Our empirical leap from 12.0% to 17.0% Pass@1 directly highlights this harness synergy: when surgical policy weights are liberated from premature turn-8 emergency fallbacks and provided a 15-minute execution runway, the agent effectively converts difficult multi-turn diagnostic sessions into verified patches.

7. CONCLUSION

We presented **PatchGRPO**, a critic-free, graph-constrained reinforcement learning framework tailored for open-weights coding agents on real-world repositories. By bounding the agent's action space to topological call graph neighborhoods, sampling candidate rollout groups, and evaluating them via deterministic compile-time and run-time reward engines, PatchGRPO overcomes credit assignment collapse and the plausible patch pathology. Our empirical findings demonstrate that PatchGRPO trains within 73.31 GB VRAM on a single A100 GPU, achieves **17.0% Pass@1** on the official competition evaluation suite (+41.7% relative gain over baseline), and uncovers the critical Generalization Horizon governing agentic RLVR.

REFERENCES

1. C. E. Jimenez, J. Yang, A. Wettig, et al. "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" In *ICLR*, 2024 (arXiv:2310.06770).
2. Gemma Team, Google DeepMind. "Gemma 4 Technical Report." *arXiv:2607.02770*, 2026.
3. R. Rafailov, A. Sharma, E. Mitchell, et al. "Direct Preference Optimization: Your Language Model is Secretly a Reward Model." In *NeurIPS*, 2023 (arXiv:2305.18290).
4. Z. Qi, F. Long, S. Achour, and M. Rinard. "An Analysis of Patch Plausibility and Correctness for Generate-and-Validate Patch Generation Systems." In *ISSTA*, pages 24–36, 2015.
5. J. Schulman, F. Wolski, P. Dhariwal, et al. "Proximal Policy Optimization Algorithms." *arXiv:1707.06347*, 2017.
6. DeepSeek-AI. "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning." *arXiv:2501.12948*, 2025.
7. Z. Shao, P. Wang, Q. Zhu, et al. "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models." *arXiv:2402.03300*, 2024.
8. E. J. Hu, Y. Shen, P. Wallis, et al. "LoRA: Low-Rank Adaptation of Large Language Models." In *ICLR*, 2022 (arXiv:2106.09685).
9. S. Yao, J. Zhao, D. Yu, et al. "ReAct: Synergizing Reasoning and Acting in Language Models." In *ICLR*, 2023 (arXiv:2210.03629).
10. N. Shinn, F. Cassano, E. Berman, et al. "Reflexion: Language Agents with Verbal Reinforcement Learning." In *NeurIPS*, 2023 (arXiv:2303.11366).
11. J. Yang, C. E. Jimenez, A. Wettig, et al. "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering." *arXiv:2405.15793*, 2024.
12. C. Xia, Y. Deng, H. Ding, et al. "Agentless: Demystifying LLM-based Software Engineering Agents." *arXiv:2407.01489*, 2024.
13. M. Wortsman, G. Ilharco, S. Y. Gadre, et al. "Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time." In *ICML*, pages 23965–23998, 2022 (arXiv:2203.05482).
14. R. Xu, C.-Y. Lee, T. Pfister, et al. "Harness²: Recursive Agent Harnessing for an Open World." *Google Cloud AI Research / arXiv:2610.14224*, 2026.